FORECASTING URBAN TRAFFIC USING IOT DATA AND ARTIFICIAL INTELLIGENCE

A Data-Driven Approach for Smart City Decision-Making

Jonathan Beck
Evan Simmons
Katya Balin
Manuel Perozo

Table of Contents

1. Abstract *3*
2. Problem Statement & Motivation *4*
3. Introduction *4*
4. Exploratory Data Analysis *5*
4.1. Summary of Findings **5**
4.2. Distribution of Traffic Volume **5**
4.3. Temporal Traffic Patterns **7**
4.4. Recurring Traffic "Hot Spots" **8**
4.5. High-Volume Roadway Segments **8**
5. Unsupervised Learning *9*
5.1. Clustering Analysis and Anomaly Detection **9**
5.2. K-Means Clustering of Traffic Profiles **9**
5.3. Cluster Interpretations **10**
5.4. Business Implications of Traffic Clustering **11**
5.5. Z-Score–Based Anomaly Detection **11**
5.6. Observations from the Anomaly Analysis **11**
5.7. Role of These Methods in the Overall Modeling Pipeline **12**
6. Data Preparation & Feature Engineering *12*
6.1. Baseline Model **12**
6.2. Regularized Linear Regression **13**
7. Baseline Model *17*
7.1. Selection Criteria **17**
7.2. Preprocessing **17**
7.3. Results **18**
8. Regularized Linear Regression Model *19*
8.1. Selection Criteria **19**
8.2. Preprocessing **20**
8.3. Design **21**
8.4. Cross-Validation Results **23**
8.5. Test Results **23**
8.5.1 Generalization 24
8.5.2 Explained Variance & Model Fit 25
8.5.3 Considerations Regarding MAPE 25
8.6 Interpretation of Coefficients & Features **26**
8.7 Diagnostic Plots **28**
8.7.1 Predicted vs Actual Traffic Volume 28
8.7.2 Residual Distribution 29
9. Business Recommendations *29*
10. Conclusion *30*

11. References **31**

1. Abstract

Advances in Internet of Things (IoT) technologies have facilitated the large-scale collection of high-frequency sensor data, creating new possibilities for data-driven decision-making in urban infrastructure and transportation. This project presents an end-to-end AI analytics framework to predict hourly traffic volumes in New York City using IoT roadway sensors. Its aim is to show how machine learning can be applied to real-world IoT data to support operational planning and business decisions in smart cities.

To account for the cyclical nature of traffic patterns, raw time variables were transformed using sine and cosine encodings to ensure continuity across hourly, daily, and monthly cycles. Features such as lag-based traffic data, a binary indicator for weekends, and roadway segment IDs were included to capture temporal dependence, behavioral variations, and location-specific patterns. A structured preprocessing pipeline was developed, employing column-wise transformations to standardize numeric data, encode categorical variables, and avoid data leakage during model training and testing. Missing values from data gaps and lag feature creation were handled by removing affected observations to preserve data integrity.

Model evaluation used a chronological train–test split to mimic real-world forecasting. A simple linear regression model showed limited predictive performance, highlighting its inadequacy for complex IoT time-series data. The final model demonstrated high accuracy on a separate test set, with a root-mean-square error of approximately 124 vehicles and a mean absolute error of approximately 84 vehicles per hour. These results indicate the method's effectiveness in providing accurate, practical forecasts.

Overall, this study illustrates how AI techniques can be integrated with IoT data pipelines to generate actionable insights, improve forecast accuracy, and support data-driven decisions in transportation and smart city applications.

2. Problem Statement & Motivation

Managing traffic in a large city like New York is an ongoing challenge. Congestion affects travel efficiency, the commuter experience, fuel consumption, and transportation agencies' ability to make timely and effective decisions. Because traffic conditions change throughout the day and vary across roadway segments, predicting hourly traffic volume offers clear practical benefits.

This project centers on predicting hourly traffic volume using historical traffic sensor data and time-based roadway patterns. Specifically, the goal is to see whether variables such as hour of day, day of week, month, roadway segment, travel direction, and recent traffic conditions can be used to accurately forecast future traffic volume. This is a complex problem, as traffic data is highly time-dependent and often exhibits cyclical rather than linear patterns.

The motivation for this work is both practical and analytical. On the practical side, better traffic predictions can help city agencies anticipate congestion, optimize roadway planning, and allocate traffic management resources more effectively. On the analytical side, this project shows how raw sensor data can be engineered, reshaped, and modeled to yield valuable predictive insights. It also emphasizes the importance of choosing modeling techniques that align with the data's structure, especially since later sections demonstrate that a simple baseline model performs significantly worse than a regularized approach.

3. Introduction

The data used in this project originates from automated traffic observations collected across roadway segments throughout New York City (Department of Transportation (DOT), 2026). The raw dataset includes hourly vehicle counts and roadway-related details, such as segment identifiers, roadway names, travel directions, and origin-to-destination indicators. These variables enable the data to reflect both when and where traffic activity occurs.

Originally, the dataset was organized in a wide format, where each row represented a roadway segment and date, with hourly traffic counts stored across many separate columns. While this structure may be useful for reporting, it is less suitable for predictive modeling. To improve usability, the data was converted to long format, with each row representing a single hourly traffic observation for a roadway segment. This transformation resulted in a time-series dataset with over one million observations.

After reshaping, unnecessary identifier fields were removed, and the remaining variables were cleaned and standardized. The final dataset included roadway segment ID, roadway name, travel direction, origin and destination indicators, date, hour, and observed traffic volume. Additional temporal features were derived from the date and hour variables. Since traffic patterns are cyclical, hour, day of the week, and month were transformed using sine and cosine functions to better represent repetitive, time-based behavior. A binary weekend variable was added, along with lagged traffic features from the previous hour, day, and week. These additions made the data more suitable for modeling by capturing both recurring traffic cycles and short-term traffic dependencies.

4. Exploratory Data Analysis

To better understand the Traffic Volume Counts (Historical) dataset from NYC Open Data, exploratory data analysis (EDA) was conducted prior to predictive modeling. The purpose of this analysis was to examine the distribution, timing, and consistency of traffic volume across the network and identify operationally meaningful patterns to inform subsequent machine learning work.

4.1. Summary of Findings

Several key findings emerged from the EDA:

- Traffic volume is highly uneven across observations, with most values falling in lower ranges and a smaller number of very high-volume observations.
- Traffic follows a strong daily cycle, with the lowest volumes overnight and the highest during late afternoon and early evening.
- Traffic remains active throughout most of the week, although Sunday volumes are noticeably lower.
- Recurring traffic "hot spots" appear at predictable times, particularly during weekday afternoon and evening periods.
- Some roadway segments consistently carry much higher traffic than others, suggesting that certain locations may warrant operational prioritization.

Overall, the EDA demonstrates that traffic volume in this dataset follows clear time-based and location-based patterns, providing a strong foundation for future predictive modeling. These findings suggest that variables such as hour of day, day of week, and roadway segment are likely to be important features in forecasting traffic behavior.

4.2. Distribution of Traffic Volume

The analysis began with a review of the statistical properties of the traffic volume variable.

summary	traffic_volume
count	1023064
mean	440.8887666265260
stddev	647.0090994462730
min	0.0
max	10532.0

Table 1 Summary statistics for traffic volume.

The summary statistics indicate substantial variation in traffic volume across observations, suggesting that the dataset captures a wide range of roadway activity levels. This is expected in a large urban environment such as New York City, where traffic differs considerably by road type, time, and surrounding activity.

A histogram was used to visualize the overall distribution of traffic volume.

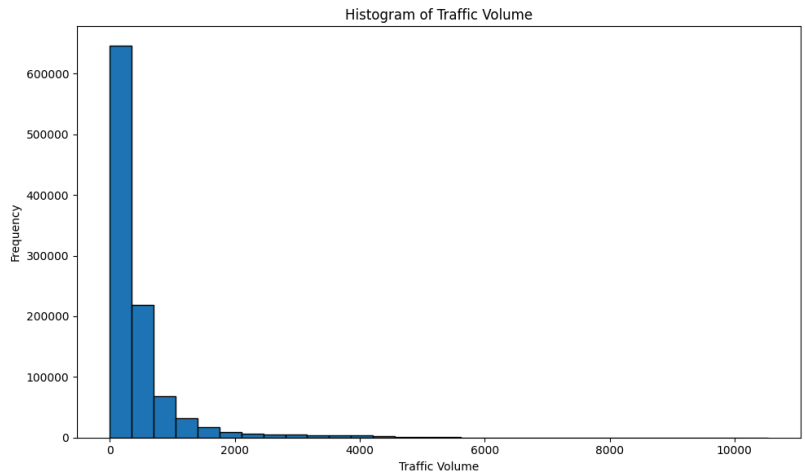

Figure 1 Histogram of traffic volume.

The histogram shows that traffic volume is strongly right-skewed, meaning that most observations fall within relatively low to moderate ranges, while a smaller number of roadway segments experience very high traffic counts. This suggests that traffic demand is not evenly distributed across the network.

A box plot was also created to further assess the spread and potential outliers.

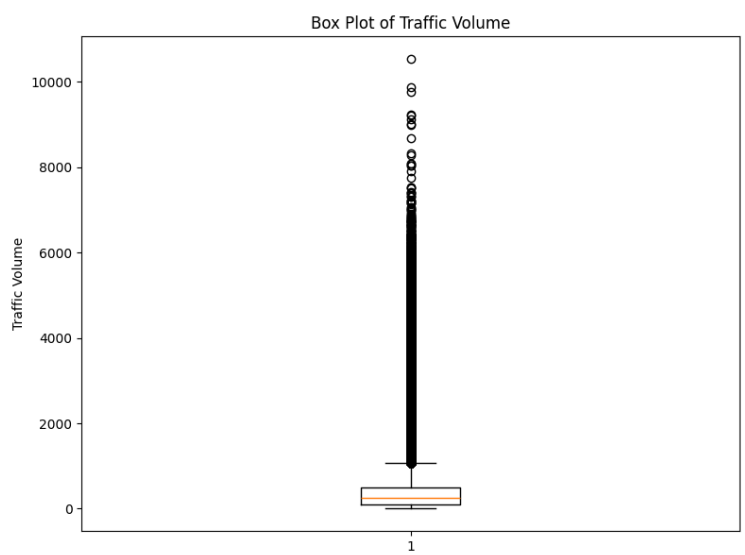

Figure 2 Box plot of traffic volume.

The box plot confirms the presence of many high-end outliers, indicating that some observations are substantially above the typical traffic range. These may reflect heavily traveled corridors, unusual congestion events, or occasional sensor irregularities. From a business perspective, these high-volume observations may represent roadway segments that warrant additional operational attention.

4.3. Temporal Traffic Patterns

To examine how traffic changes over time, average traffic volume was analyzed by hour of day.

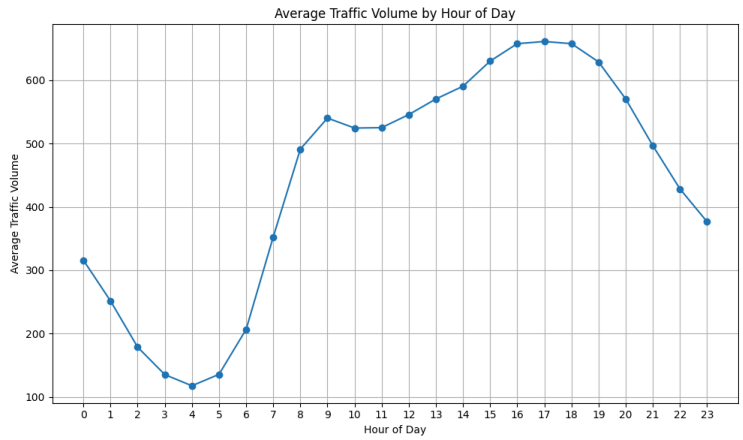

Figure 3 Average traffic volume by hour of day.

The hourly trend reveals a clear daily traffic cycle. Traffic is lowest in the early morning, begins increasing after approximately 6:00 AM, remains elevated throughout the day, and reaches its highest levels in the late afternoon and early evening. This pattern is consistent with expected commuter and daytime urban activity.

From an operational standpoint, this suggests that traffic demand follows predictable daily cycles, which is useful for identifying peak congestion windows and informing future forecasting efforts.
Traffic was also examined by day of the week.

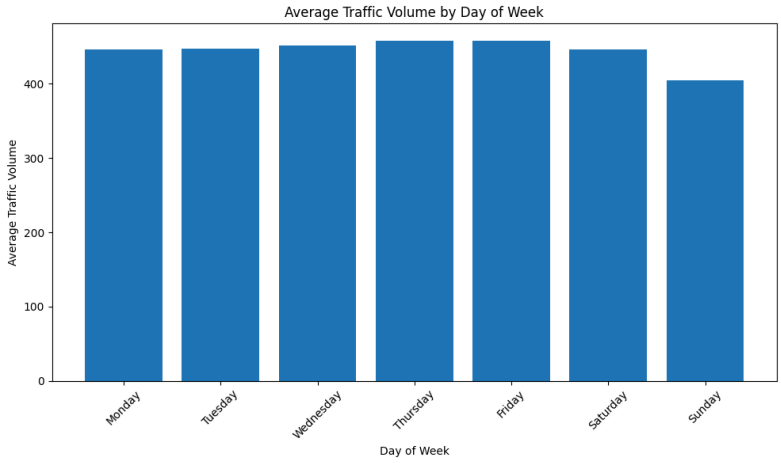

Figure 4 Average traffic volume by day of week.

The day-of-week analysis shows that traffic remains relatively stable from Monday through Saturday, while Sunday exhibits noticeably lower average traffic volume. This suggests that traffic demand in New York City remains consistently active for most of the week and is not limited only to weekday

commuting. The lower Sunday volumes are consistent with reduced work-related travel and business activity.

4.4. Recurring Traffic "Hot Spots"

To provide a more compact view of temporal traffic behavior, a heat map was created using average traffic volume by day of week and hour of day.

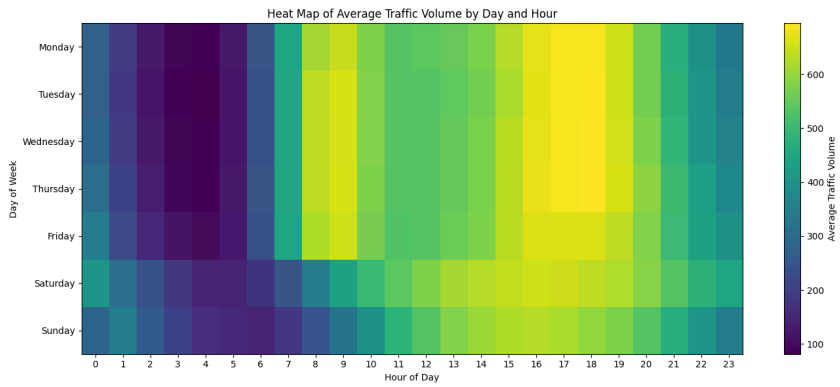

Figure 5 Heat map of average traffic volume by day and hour.

The heat map clearly highlights recurring traffic "hot spots" across the week. Weekday traffic patterns are highly consistent, with the highest concentrations occurring during the late afternoon and early evening.

Weekend traffic follows a different pattern, beginning later in the day and remaining active into the evening.
This visualization is especially useful because it shows that traffic demand is not random but follows structured, recurring temporal patterns. This is important for both traffic operations and future machine learning applications.

4.5. High-Volume Roadway Segments

To identify roadway segments that may require greater operational prioritization, the top high-volume locations were examined.

segment_id	roadway__name	from	to	direction	avg_traffic_volume
193991	LONG ISLAND EXPRESSWAY	73 PLACE	74 STREET	WB	4419.80
142386	LONG ISLAND EXPRESSWAY	JUNCTION BOULEVARD	97 STREET	EB	4385.19
139303	BRONX RIVER PARKWAY	BOSTON ROAD	ENTRANCE	SB	4376.39
193992	LONG ISLAND EXPRESSWAY	73 PLACE	74 STREET	EB	4239.77
9012150	GRAND CENTRAL PARKWAY	64 AVENUE PEDESTRIAN OVERPASS	64 ROAD	SB	4163.16

192292	2 AVENUE	EAST 59 STREET	EAST 60 STREET	SB	4059.12
139657	BRONX RIVER PARKWAY	MORRIS PARK AVENUE	BRONXDALE AVENUE	SB	4058.77
137523	LONG ISLAND EXPRESSWAY	57 AVENUE	254 STREET	WB	3943.19
137522	LONG ISLAND EXPRESSWAY	57 AVENUE	254 STREET	EB	3896.27
138288	Van Wyck Expressway	105 Avenue	LibertyAvenue	SB	3838.19

Table 2 Top 10 roadway segments by average traffic volume.

This table highlights the roadway segments that consistently carry the greatest traffic demand. These locations likely represent critical mobility corridors and may be especially important for congestion management, infrastructure planning, and predictive modeling. Identifying these high-volume segments supports a more targeted and business-relevant approach to traffic analysis.

While the exploratory data analysis identified important time-based and location-based traffic patterns, it did not determine if roadway segments could be grouped by similar traffic behaviors. The next step uses K-Means clustering to segment observations into traffic pattern groups, enabling us to see whether roads or intersections share characteristics such as heavy commuter demand, low traffic, or high activity throughout the day. This segmentation helps tailor traffic management strategies, rather than treating all segments the same.

5. Unsupervised Learning

5.1. Clustering Analysis and Anomaly Detection

As outlined in our initial project proposal, our team examined the Traffic Volume Counts (Historical) dataset from NYC Open Data using various exploratory data analysis (EDA) techniques to better understand traffic behavior across New York City's roadway network. Specifically, we implemented K-Means clustering to identify distinct traffic patterns and Z-score–based anomaly detection to spot unusual traffic events. These methods helped uncover underlying traffic trends and potential disruptions that could impact operational decisions and predictive modeling.

The main goal of this analysis was to segment roads and intersections with similar traffic behavior and to detect abnormal traffic volumes that might indicate disruptions such as construction, accidents, or signal malfunctions.

5.2. K-Means Clustering of Traffic Profiles

To understand daily traffic behavior, we used K-Means clustering on hourly traffic data. K-Means groups observations by similarity, here to identify locations with similar 24-hour traffic patterns. It found four clusters, each with a different traffic profile. While overall patterns were similar, clusters differed mainly by traffic intensity and peaks. Consistent patterns across clusters included: lowest volumes late night/early morning (0–5), sharp increase around 6–8 AM, stabilization midday (10–14),

a late afternoon/early evening peak (4–6 PM), and a gradual decline late evening. These patterns reflect typical urban commuting, driven by work and economic activity.

5.3. Cluster Interpretations

Cluster	Traffic Volume Level	Typical Daily Pattern	Likely Road Type	Operational / Business Implications
Cluster 0	Low traffic volume	Small increases during morning and evening commute periods; overall low activity throughout the day	Residential streets, local roads	Requires minimal traffic management. Monitoring may focus on localized disruptions such as construction or school-related traffic.
Cluster 1	Moderate traffic volume	Clear morning and evening commuting peaks with moderate midday activity	Secondary arterial roads or neighborhood connectors	Important for distributing traffic between residential areas and major corridors. May benefit from optimized traffic signal timing.
Cluster 2	High traffic volume	High traffic levels throughout the day with strong evening peaks and sustained midday traffic	Major arterial roads, highways, commuter routes	Critical infrastructure requiring advanced monitoring, congestion mitigation strategies, and potential adaptive traffic management.
Cluster 3	Medium-high traffic volume	Pronounced commuting peaks with sustained activity during midday hours	Commercial corridors, busy urban avenues	Supports both commuting and commercial activity. Traffic management strategies must balance congestion reduction with economic activity.

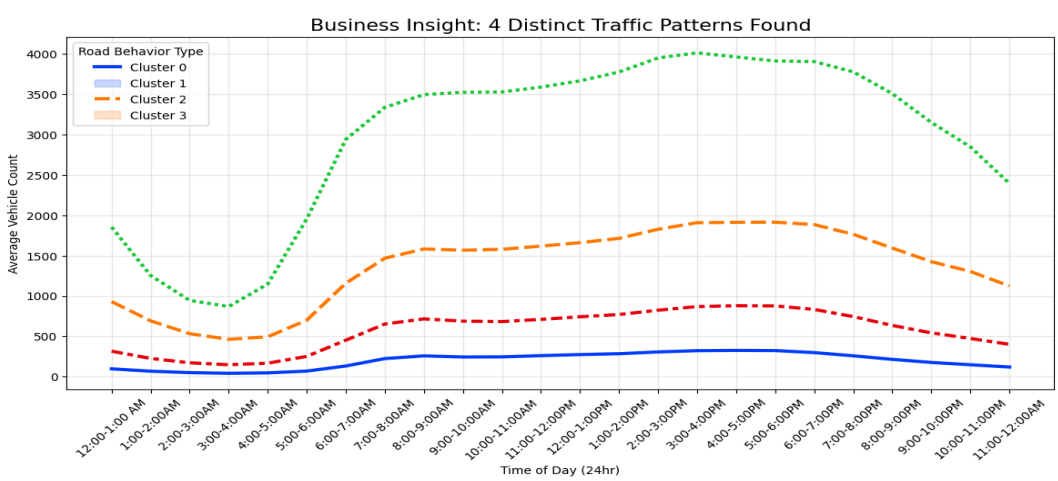

Figure 6 Average hourly traffic volume by cluster, illustrating distinct traffic intensity levels and daily commuting patterns.

5.4. Business Implications of Traffic Clustering

The clustering results show that roads should not all be managed the same. Grouping roads by traffic behavior enables targeted strategies. Low-volume roads need minimal intervention, while high-volume arterials require advanced monitoring. This segmentation helps planners allocate resources and tailor policies to each road type.

5.5. Z-Score–Based Anomaly Detection

In addition to clustering, we implemented Z-score anomaly detection to identify unusually high or low traffic volumes relative to typical patterns.

The Z-score measures how far a given observation deviates from the mean: $Z = (X - \mu) / \sigma$	Where: X = observed traffic volume μ = average traffic volume σ = standard deviation

Observations with $|Z| > 3$ were flagged as anomalies, indicating traffic volumes that are statistically unusual compared to historical patterns.

5.6. Observations from the Anomaly Analysis

The anomaly detection showed most traffic was normal, but some unusual events were found, especially early mornings when low baseline traffic makes small increases seem large. Possible causes include road closures, construction, accidents, special events, or signal issues. Fewer anomalies occur during peak hours, but those that do could indicate major disruptions.

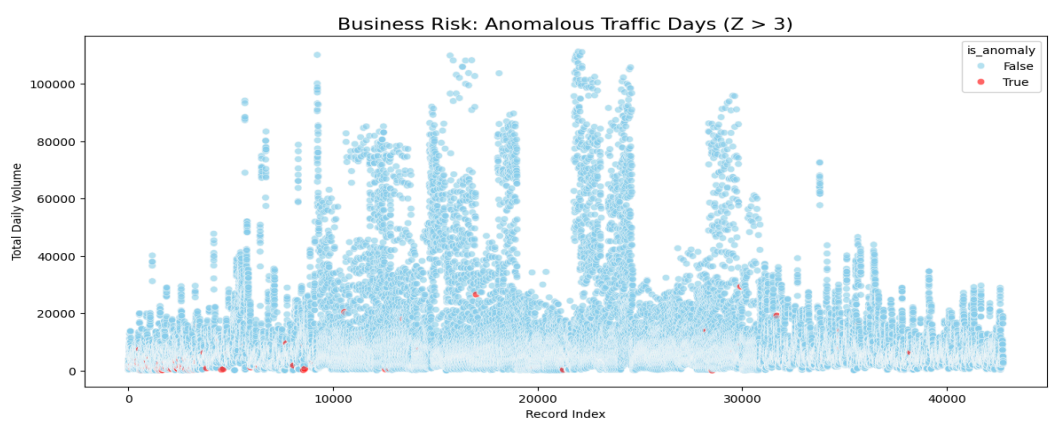

Figure 7 Traffic anomaly detection using Z-scores (|Z| > 3). Red points indicate traffic volumes that deviate significantly from historical expectations.

5.7. Role of These Methods in the Overall Modeling Pipeline

Both clustering and anomaly detection offered valuable insights early in the data analysis. Clustering identified traffic categories in the roadway network, aiding feature engineering and model segmentation for future predictions. Anomaly detection spotted rare events that suggest disruptions, supporting real-time monitoring systems. Together, these techniques deepened understanding of the dataset and laid the foundation for developing models to forecast traffic and identify congestion risks across the network.

6. Data Preparation & Feature Engineering

6.1. Baseline Model

Following data preparation and feature engineering, a baseline linear regression model was developed to establish an initial benchmark for predicting hourly traffic volume. While the transformed dataset is well-suited for modeling, it is important to first evaluate how effectively a simple and interpretable model can capture the underlying patterns in the data before introducing more advanced techniques.

To construct this baseline model, the engineered feature set described in the previous section was utilized. This includes temporal variables representing hourly, daily, and monthly patterns, a binary **is_weekend** indicator, and directional information capturing roadway orientation. These predictors were selected to reflect fundamental temporal and spatial patterns in traffic behavior observed throughout the dataset **(Figure 8)**.

Given that the dataset had already been reshaped, cleaned, and reduced to relevant predictors, no additional model complexity was introduced at this stage. The baseline model was intentionally kept simple, allowing for a clear evaluation of how well linear relationships alone can explain variation in traffic volume across roadway segments and time periods **(Figure 8)**.

```
display(df.select(
    "traffic_volume",
    "hour",
    "weekday",
    "month",
    "is_weekend",
    "direction"
).limit(10))
```

See performance (1)

Table +

	1.2 traffic_volume	1²3 hour	1²3 weekday	1²3 m...	is_weekend	direction
1	20	1	0	1	false	NB
2	21	1	1	1	false	NB
3	27	1	2	1	false	NB
4	22	1	3	1	false	NB
5	31	1	4	1	false	NB
6	42	1	5	1	true	NB
7	27	1	0	1	false	SB
8	26	1	1	1	false	SB
9	32	1	2	1	false	SB
10	24	1	3	1	false	SB

Figure 8 Dataset Table Columns

Ultimately, this baseline model serves as a foundational reference point within the modeling pipeline. Its performance provides insight into the limitations of basic linear approaches and highlights the necessity for more advanced methods, such as regularized regression, which are explored in the following section.

6.2. Regularized Linear Regression

In designing a high-quality predictive model for NYC traffic volume, the Automated Traffic data was transformed into a structured time-series format suitable for supervised learning and plotting. The raw dataset records hourly vehicle counts across numerous roadway segments in NYC; however, it required several preprocessing and feature engineering steps to make it more actionable for granular predictive modeling and analytics.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42756 entries, 0 to 42755
Data columns (total 32 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   42756 non-null  int64
 1   SegmentID            42756 non-null  int64
 2   Roadway Name         42756 non-null  object
 3   From                 42756 non-null  object
 4   To                   42756 non-null  object
 5   Direction            42756 non-null  object
 6   Date                 42756 non-null  object
 7   12:00-1:00 AM        42752 non-null  float64
 8   1:00-2:00AM          42752 non-null  float64
 9   2:00-3:00AM          42752 non-null  float64
 10  3:00-4:00AM          42752 non-null  float64
 11  4:00-5:00AM          42752 non-null  float64
 12  5:00-6:00AM          42752 non-null  float64
 13  6:00-7:00AM          42752 non-null  float64
 14  7:00-8:00AM          42752 non-null  float64
 15  8:00-9:00AM          42752 non-null  float64
 16  9:00-10:00AM         42752 non-null  float64
 17  10:00-11:00AM        42753 non-null  float64
 18  11:00-12:00PM        42755 non-null  float64
 19  12:00-1:00PM         42503 non-null  float64
 20  1:00-2:00PM          42503 non-null  float64
 21  2:00-3:00PM          42503 non-null  float64
 22  3:00-4:00PM          42503 non-null  float64
 23  4:00-5:00PM          42503 non-null  float64
 24  5:00-6:00PM          42503 non-null  float64
 25  6:00-7:00PM          42503 non-null  float64
 26  7:00-8:00PM          42503 non-null  float64
 27  8:00-9:00PM          42503 non-null  float64
 28  9:00-10:00PM         42503 non-null  float64
 29  10:00-11:00PM        42503 non-null  float64
 30  11:00-12:00AM        42503 non-null  float64
 31  Total_Daily_Traffic  42756 non-null  float64
dtypes: float64(25), int64(2), object(5)
memory usage: 10.4+ MB
```

Figure 9 Daily hourly intervals

To begin, the dataset was originally structured in a wide format, in which each observation represented a specific roadway segment and date, while the hourly traffic counts were stored across multiple columns corresponding to different hourly intervals throughout the day (Figure 9). While this structure is convenient for manual reporting, it isn't optimal for predictive modeling because machine learning algorithms typically expect each observation to correspond to a single instance of the target variable (hourly traffic volume).

As such, the first step was to reshape the data into long format by merging the hourly traffic columns into a single variable representing traffic volume for each roadway segment at each hour.

To do this, we applied a melt operation, in which roadway identifiers and location descriptors (such as segment ID, roadway name, and direction information) were retained as identifier variables, while the hourly columns were converted into two new fields: an hour dimension and a **traffic_volume** dimension (Figure 10).

```
hour_cols = [hour for hour in df.columns if ":" in hour]
```

```
df = df.melt(
    id_vars=["ID", "SegmentID","Roadway Name","From","To","Direction"
    value_vars=hour_cols,
    var_name="hour",
    value_name="traffic_volume"
)
```

Figure 10 Data reshaping with "melt"

This significantly increased the dataset size, resulting in a time-series dataset with over 1 million observations per hour (Figure 11).

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1026144 entries, 0 to 1026143
Data columns (total 8 columns):
 #   Column          Non-Null Count    Dtype
---  ------          --------------    -----
 0   SegmentID       1026144 non-null  int64
 1   Roadway Name    1026144 non-null  object
 2   From            1026144 non-null  object
 3   To              1026144 non-null  object
 4   Direction       1026144 non-null  object
 5   Date            1026144 non-null  object
 6   hour            1026144 non-null  object
 7   traffic_volume  1023064 non-null  float64
dtypes: float64(1), int64(1), object(6)
memory usage: 62.6+ MB
```

Figure 11 Dataset size after "melt"

Following the transformation, unnecessary columns, such as row-identification (ID) columns, were removed, and all column names were normalized to simplify downstream analysis and functions. After this initial round of feature selection, we had a dataset with eight dimensions: roadway segment identifier, roadway name, directional information, origin and destination indicators (from and to), date, hour, and observed traffic volume.

Then, because traffic behavior is cyclical and continuous by nature, traditional time-variable transformations (e.g., representing hour values on the 0-23 scale) can introduce unnatural discontinuities in the data. For example, the hours 11:00 PM (23) and 12:00 AM (0) are adjacent, similar time periods, yet they are treated as distant, dissimilar, and discontinuous in traditional linear modeling. To correct this issue, the derived hour, day of week (0-6), and month (0-11) fields were each transformed using sine and cosine pairings, allowing the model to estimate continuous, cyclical periodic traffic patterns across time, such as hourly commuting behaviors and day-to-day and month-to-month traffic fluctuations (Figure 12).

```
df = df.assign(
  hour_sin = lambda x: np.sin(2 * np.pi * x.hour / 24),
  hour_cos = lambda x: np.cos( 2 * np.pi * x.hour / 24),
  dow_sin = lambda x: np.sin(2 * np.pi * x.weekday / 7),
  dow_cos = lambda x: np.cos(2 * np.pi * df.weekday / 7),
  month_sin = lambda x: np.sin(2 * np.pi * x.month / 12),
  month_cos = lambda x: np.cos(2 * np.pi * x.month / 12)
)
```

Figure 12 Cyclical Time Encoding Using Sine–Cosine Transformations

We did, however, decide to incorporate a binary **is_weekend** explanatory indicator, as our EDA endeavors foreshadowed that traffic volumes on weekend days (Saturday and Sunday) appeared less voluminous compared to the average weekday observation; thus, we believed incorporating this feature in modeling could help explain some of the variance in our response variable (Figure 13).

```
X = X.assign(
    is_weekend = lambda x: np.where(x.is_weekend==True, 1, 0)
)
```

Figure 13 Binary Weekend Classification

In addition, to capture how recent traffic conditions tend to influence "current" traffic demand patterns, several lag-based predictor variables were engineered. Specifically, we decided to include traffic volume from the previous hour, the same hour on the previous day, and the same hour during the previous week as explanatory variables in the model. Including these lag features allowed the model to account for the cyclical nature often characterizing urban traffic data, in which roadway behaviors tend to change slowly and follow deterministic, consistent daily and weekly patterns (Figure 14).

```
df = df.sort_values(['segment_id', 'timestamp'])

df = df.assign(
    lag_1h = lambda x: x.groupby('segment_id').traffic_volume.shift(1), # last hour traffic
    lag_24h = lambda x: x.groupby('segment_id').traffic_volume.shift(24), # traffic same hour yesterday
    lag_168h = lambda x: x.groupby('segment_id').traffic_volume.shift(168) # traffic same hour last week

)
```

Figure 14 Lag-Based Traffic Volume Features

Following these transformations, it was decided that a handful of the explanatory variables could now be dropped, as they introduced unnecessary noise and needlessly inflated dimensionality. For example, raw date components were removed once their temporal structure had already been extracted, and string-heavy roadway fields were removed, as keeping them would have introduced an excessive number of sparse dummy variables, given their information was already captured more efficiently through the use of (roadway) segment identifiers. Ultimately, this final feature selection process significantly improved model stability while preserving the key temporal and spatial signals required to accurately forecast traffic behavior in NYC (Figure 15).

```
X = df.drop(
    columns = ['date', # already extracted necessary components
               'hour_start', # string representation of hour
               'day_of_week', # duplicates info
               'hour',
               'weekday',
               'month', # have cyclical variables
               'roadway__name',
               'from',
               'to' # these have thousands of unique values and would explode dummy variables + redundant with segment_id
               ]
)
```

Figure 15 Removing Redundant Features to Improve Model Stability

7. Baseline Model

7.1. Selection Criteria

To establish a benchmark for traffic volume prediction, a standard linear regression model was selected as the baseline. Linear regression was chosen for its simplicity, interpretability, and ability to provide a clear understanding of how individual features contribute to predicting traffic volume.

The primary objective of implementing the baseline model is not to achieve optimal predictive performance, but rather to evaluate how effectively a simple linear relationship can capture variation in the response variable. Given the structured time-series nature of the dataset and the inclusion of engineered temporal and lag-based features, linear regression provides a logical and interpretable starting point for assessing model performance.

More complex modeling approaches, such as decision trees or other non-linear methods, were intentionally excluded at this stage to preserve interpretability and establish a meaningful point of comparison. Beginning with a baseline linear model makes it possible to clearly quantify the incremental improvements achieved by more advanced techniques introduced later in the analysis.

7.2. Preprocessing

Prior to training the baseline model, several preprocessing steps were applied to ensure that the dataset was suitable for linear modeling. While most data transformation and feature engineering steps were completed in the previous section, additional preprocessing was necessary to prepare the data for model training.

First, observations containing missing values were removed from the dataset. These missing values were primarily introduced during the construction of lag-based features, as earlier observations within each roadway segment lack sufficient historical data. Given the size of the dataset, removing these observations was preferred over imputation, as it avoids introducing artificial values that could distort model behavior.

Next, the dataset was partitioned into training and test sets to support model evaluation. A randomized split was applied, allocating 70% of the data for training and 30% for testing. This approach ensures that the model is evaluated on unseen data, providing a general measure of predictive performance. The split was performed using a fixed seed to maintain reproducibility of results (Figure 16).

```
12:25 PM (4s)
train_data, test_data = final_data.randomSplit([0.7, 0.3], seed=42)

print("Training rows:", train_data.count())
print("Testing rows:", test_data.count())
See performance (2)
test_data: pyspark.sql.connect.dataframe.DataFrame
train_data: pyspark.sql.connect.dataframe.DataFrame
Training rows: 716559
Testing rows: 306505
```

Figure 16 70/30 Training–Test Split for Model Validation

Finally, the feature set was organized into predictor and response variables, with traffic volume as the target variable and all engineered features as explanatory inputs. No additional scaling or regularization techniques were applied in the baseline model, as the goal was to evaluate raw linear relationships without introducing further transformations.

7.3. Results

The baseline linear regression model demonstrated limited predictive performance, indicating that simple linear relationships alone are insufficient to fully capture the complexity of traffic behavior in New York City.

On the test dataset, the model achieved an R-squared value of approximately 0.05, suggesting that only about 5% of the variance in hourly traffic volume is explained by the model's feature set. This result highlights that most of the variability in traffic patterns is driven by factors not effectively captured by basic linear modeling.

Additionally, the model produced relatively high prediction error, with an RMSE of approximately 631.75 and an MAE of approximately 360.12. These error metrics indicate that, on average, the model's predictions deviate from actual traffic counts by several hundred vehicles per hour, suggesting that the baseline model is not sufficiently accurate for reliable traffic forecasting (Figure 17).

```python
from pyspark.ml.evaluation import RegressionEvaluator

rmse_eval = RegressionEvaluator(
    labelCol="traffic_volume",
    predictionCol="prediction",
    metricName="rmse"
)

r2_eval = RegressionEvaluator(
    labelCol="traffic_volume",
    predictionCol="prediction",
    metricName="r2"
)

mae_eval = RegressionEvaluator(
    labelCol="traffic_volume",
    predictionCol="prediction",
    metricName="mae"
)

lr_rmse = rmse_eval.evaluate(lr_pred)
lr_r2 = r2_eval.evaluate(lr_pred)
lr_mae = mae_eval.evaluate(lr_pred)

print("Baseline Linear Regression Results")
print("RMSE:", lr_rmse)
print("MAE:", lr_mae)
print("R2:", lr_r2)
```

```
Baseline Linear Regression Results
RMSE: 631.7490335108082
MAE: 360.12476963670366
R2: 0.050604041176304015
```

Figure 17 Performance Metrics for Baseline Traffic Model

From an interpretability standpoint, the model coefficients indicate that temporal features such as hour and month contribute to traffic predictions, while the negative coefficient for the **is_weekend** variable reflects reduced traffic volume on weekends. Although these relationships align with expected traffic patterns, they remain insufficient to explain overall variation in the data.

Overall, the baseline model serves its intended purpose as a reference point rather than a high-performing predictive tool. The low explanatory power and high error rates clearly demonstrate the limitations of simple linear regression in this context and motivate the need for more advanced modeling approaches, which are explored in the subsequent section.

8. Regularized Linear Regression Model

8.1. Selection Criteria

To build an interpretable yet practical supervised linear model, Elastic Net regularization was selected as the optimal modeling tool. Elastic Net combines the strengths of Ridge and Lasso regression by simultaneously applying a fractional blend of both L1 (Lasso) and L2 (Ridge) penalties. This approach provides two key advantages in the context of traffic prediction: it stabilizes coefficient estimates when predictors are highly correlated (L2 Penalty) and performs implicit feature selection by shrinking less influential variables toward zero (L1 Penalty).

The traffic prediction problem has several characteristics that make the application of regularization particularly effective. First, the time-related features, such as lagged traffic volumes and cyclical time variables, are naturally correlated. Secondly, the inclusion of numerous roadway segment identifiers introduces high-dimensional categorical features. Elastic Net regularization addresses these discrepancies by balancing coefficient shrinkage with feature retention, resulting in a stable yet interpretable model.

8.2. Preprocessing

Naturally, we had to determine an efficient way to handle sporadic null values across the data. For one, there were quite a few observations with missing values in their hourly traffic fields. Moreover, introducing lag-based predictors as explanatory fields naturally induces missing values in this dimension at the beginning of each roadway segment's time series, since earlier observations don't always exist. Therefore, given the size of our dataset, rows containing these missing features were removed from the modeling dataset. Imputing these fields would have required artificially generating synthetic traffic values across the dataset that didn't occur, potentially weakening the reliability of our predictive model. Thus, simply dropping these observations avoids this possibility entirely and keeps the realism and truth of our dataset intact **(Figure 18)**.

```
X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1026144 entries, 996039 to 973885
Data columns (total 14 columns):
 #   Column          Non-Null Count    Dtype
---  ------          --------------    -----
 0   segment_id      1026144 non-null  int64
 1   direction       1026144 non-null  object
 2   is_weekend      1026144 non-null  int64
 3   timestamp       1026144 non-null  object
 4   traffic_volume  1023064 non-null  float64
 5   hour_sin        1026144 non-null  float64
 6   hour_cos        1026144 non-null  float64
 7   dow_sin         1026144 non-null  float64
 8   dow_cos         1026144 non-null  float64
 9   month_sin       1026144 non-null  float64
 10  month_cos       1026144 non-null  float64
 11  lag_1h          1021179 non-null  float64
 12  lag_24h         977512 non-null   float64
 13  lag_168h        698272 non-null   float64
dtypes: float64(10), int64(2), object(2)
memory usage: 117.4+ MB
```

```
X = X.dropna(subset=['traffic_volume','lag_1h', 'lag_24h', 'lag_168h'])
```

Figure 18 Handling Missing Traffic Observations

Because the dataset is a time series of traffic observations, a traditional random split into training and test sets wouldn't be appropriate. Doing so would allow the model to train on observations that occur after the testing period (which is supposed to represent future, unseen/unavailable data points). Thus, this would induce data leakage, where information from future traffic patterns could inadvertently skew the validity of model training. To prevent this issue and better reflect real-world conditions, the dataset was split chronologically (Figure 19). Earlier observations were used to train and tune the model, while the most recent periods were used for testing. This approach mirrors the operational setting in which real-time series predictive models, including traffic-related models, are deployed: these models learn from historical traffic data and are then used to predict conditions in future time periods.

```
train = X.loc[X.timestamp <= "2019-12-01"]

test = X.loc[X.timestamp > "2019-12-01"]
```

```
print(len(train))
print(len(test))
```

```
555948
137856
```

Figure 19 Temporal Train–Test Split

Once the dataset was cleaned and structured, a preprocessing pipeline was constructed to transform the predictor variables into a format suitable for model training. Because variables in a feature set tend to differ drastically in scale, a column-wise preprocessing pipeline was implemented using the ColumnTransformer class. Numeric lag variables were standardized using StandardScaler, to ensure that the differences in magnitude between the lag variables and the rest of the predictors would not distort coefficient estimation. Categorical variables such as roadway segment identifies and travel direction were transformed using the OneHotEncoder class to allow the model to capture location-specific traffic patterns. The remaining predictors (cyclical features and is_weekend) were passed through the pipeline unchanged, as they are already naturally scaled within bounded ranges. Ultimately, combining these transformations into a unified, structured pipeline ensured that all preprocessing steps were applied consistently during both training and prediction, and prevented information leakage between training and validation data during hyperparameter tuning (Figure 20).

```
categorical_features = ['segment_id', 'direction']
numeric_features = ['lag_1h', 'lag_24h', 'lag_168h']

numeric_transformer = Pipeline([
  ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
  ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
  ('num', numeric_transformer, numeric_features),
  ('cat', categorical_transformer, categorical_features)
], remainder='passthrough') # bypasses normalization of features (cyclical and binary) since they are already scaled naturally
```

```
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', ElasticNet(max_iter=10000, tol=1e-3))
])
```

Figure 20 Preprocessing Pipeline for Model Training

8.3. Design

To determine the optimal model configuration, hyperparameters were tuned using grid search with time-series cross-validation **(Figure 21)**.

```
grid = GridSearchCV(
  estimator=pipeline,
  param_grid=param_grid,
  cv=tscv,
  scoring='neg_root_mean_squared_error',
  n_jobs=-1 # uses all CPU cores (speeds up training)
)
```

Figure 21 Optimal model configuration using GridSearchCV

A five-fold TimeSeriesSplit procedure was used to evaluate candidate models. Unlike traditional k-fold cross-validation, this method preserves chronological ordering (Figure 22). Each fold trains the model on earlier time periods and evaluates performance on later "simulated future" periods within the fold. This structure ensures that validation results accurately represent the model's ability to generalize to future unknown traffic conditions.

```
tscv = TimeSeriesSplit(n_splits=5)
```

Figure 22 Five-fold time series split

Specifically, our grid search algorithm was used to evaluate the combinations of two critical Elastic Net Hyperparameters:

- *Alpha*: the overall strength of regularization applied to the model
- *L1 ratio*: the balance between Lasso (L1) and Ridge (L2) penalties; with 0.0 being full Ridge, 1.0 being full Lasso, and 0.5 being a perfect blend of the two.

The search space included several candidate values for each parameter, allowing the model to identify the optimal configuration that minimized prediction error across all five folds of the validation split (Figure 23).

```
param_grid = {
  'model__alpha': [0.01, 0.1, 1, 10],
  'model__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]
}
```

Figure 23 Elastic Net Hyperparameters

Thus, the combination of parameters that returned the lowest average prediction error across all five validation splits of the five folds was our best deployable model for predicting on the true holdout test set (Figure 24).

	param_model__alpha	param_model__l1_ratio	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score
0	0.01	0.1	173.320463	253.854028	165.731158	197.926422	239.683311
1	0.01	0.3	176.056195	254.024716	165.742940	197.946356	239.404367
2	0.01	0.5	180.632846	254.152738	165.776792	198.071347	238.996430
3	0.01	0.7	189.582597	254.182669	165.884844	198.566586	238.295426
4	0.01	0.9	214.345617	253.922707	166.604001	202.243786	236.748405
5	0.10	0.1	168.818527	247.101444	166.719336	203.101548	244.430488
6	0.10	0.3	167.335622	248.418371	166.359810	201.199056	243.180866
7	0.10	0.5	166.163265	249.996623	166.052961	199.536276	242.051551
8	0.10	0.7	165.683268	251.887107	165.814659	198.139260	241.023610
9	0.10	0.9	168.802968	254.157219	165.673164	196.859409	239.807882
10	1.00	0.1	249.517427	250.470746	187.211276	312.453617	322.101702
11	1.00	0.3	231.897279	246.540826	182.610899	287.085725	302.686231
12	1.00	0.5	211.875690	243.353135	177.536146	259.399739	282.476815
13	1.00	0.7	189.836021	242.199892	172.096321	230.380778	262.483438
14	1.00	0.9	168.313273	247.105046	167.039061	204.185437	245.387029
15	10.00	0.1	417.628537	326.934349	241.392004	604.159670	591.693245
16	10.00	0.3	406.613657	319.268352	236.818445	580.615655	566.955104
17	10.00	0.5	388.855185	307.849286	229.824406	544.044800	529.419382
18	10.00	0.7	355.573919	289.247035	217.807739	479.629546	465.947552
19	10.00	0.9	271.342743	256.415736	192.295820	337.634144	339.932943

Figure 24 Validation split test scores

8.4. Cross-Validation Results

The optimal configuration settings identified for the Elastic Net model were:

- *Alpha = 0.1*
- *L1 ratio = 0.7*

These settings describe a model that behaves primarily like a Lasso regression, meaning it will drive coefficients to zero when weaker signals are present, and retains some Ridge characteristics to manage correlated predictors and stabilize the model. In turn, with these hyperparameter settings, the model achieves a cross-validated average RMSE of ~204 vehicles, providing an efficient quantitative benchmark for the model's predictive accuracy when applied to true, unseen future periods **(Figure 25)**.

```
print(f'Best hyperparameters: {grid.best_params_}')
print(f'Best CV RMSE: {-grid.best_score_}')
```

```
Best hyperparameters: {'model__alpha': 0.1, 'model__l1_ratio': 0.7}
Best CV RMSE: 204.50958073647834
```

Figure 25 Optimal configuration settings

8.5. Test Results

On the training data, the model achieved an RMSE of 191.8, an MAE of 109, a MAPE of 93.8%, an R-squared of 85.3%, and an adjusted R-squared of 85.2%. On the test dataset, the model achieved an RMSE of 124.4, an MAE of 84, a MAPE of 110.5%, an R-squared of 77.9%, and an adjusted R-squared of 77.6% **(Figure 26)**.

	dataset	RMSE	MAE	MAPE (%)	R2	Adjusted R2
0	train	191.825303	109.048867	93.782433	0.852556	0.852055
1	test	124.392533	84.032082	110.501377	0.779430	0.776376

Figure 26 Elastic Net Model Performance Metrics

The test RMSE of ~124 vehicles indicates that, on average, the model's predictions deviate from actual traffic counts by roughly this value. Additionally, the MAE of ~84 vehicles provides perhaps a more interpretable measure of typical prediction error. In practical terms, this metric indicates that most hourly traffic predictions on the holdout test set fall within +/- 84 vehicles of the true observed value.

Considering the range and skew of traffic volume counts in the NYC data, these returns are impressive (Figure 27). We believe this level of accuracy makes the model acceptable and enables meaningful decision-making and planning processes to guide traffic operations.

```
X.traffic_volume.describe()

count    693804.000000
mean        358.894136
std         464.735262
min           0.000000
25%          98.000000
50%         235.000000
75%         441.000000
max        9211.000000
Name: traffic_volume, dtype: float64
```

Figure 27 NYC dataset statistics

8.5.1 Generalization

Comparing the training and test performance metrics indicates that the model generalizes well to unseen data. While the training R-squared of ~0.85 is higher than the test's R-squared (~0.78), the difference is moderate, which implies the model was able to capture true traffic behaviors.

Interestingly, both RMSE and MAE are lower on the test data than on the training data. While this may seem counterintuitive, it can occur, especially when using regularized models such as Elastic Net. The regularization process naturally prevents the coefficient weights from becoming unnecessarily large, which helps the model avoid overfitting to noise in the training data and improves its ability to generalize.

Additionally, differences in traffic patterns between the training and test periods may contribute to this effect. For instance, if the training dataset contains more extreme traffic values, it may naturally produce higher average errors than during a more stable test period.

Overall, the model demonstrates strong generalization and maintains predictive performance on unseen observations.

8.5.2 Explained Variance & Model Fit

The model attained a test R-squared of ~0.78, which suggests that roughly 78% of the variance in hourly traffic volume is explained by the model's feature set. Furthermore, this indicates strong explanatory power, given the cyclical nature of traffic behavior.

To further assess model fit, the adjusted R-squared metric was also evaluated. This statistic penalizes the model for including arbitrary predictors that don't meaningfully improve predictive performance. This was particularly important to gauge, given that even after regularization, the final model retained 1,883 predictors **(Figure 28)**.

```
# number of fitted predictors after fitting
n_features = len(feature_names)
print(f'Number of kept features in model: {n_features}')

Number of kept features in model: 1883
```

Figure 28 Elastic Net Kept Features

Impressively, the model achieved an adjusted R-squared of ~0.776 on the test set.

The small difference between these two statistics suggests that the features kept in the model do influence traffic volume in NYC, to some extent. If the variables in the feature set were highly correlated or largely uninformative, the adjusted R-squared would be substantially lower than the R-squared metric.

This result highlights an important strength of the Elastic Net modeling technique. Although the feature space is high-dimensional, regularization effectively constrains the magnitude of coefficients and limits overfitting. As a result, the model maintains strong explanatory power while avoiding the common issues associated with fitting a model using a large number of predictors.

Ultimately, the combination of a high R-squared and a similar adjusted R-squared provides convincing evidence that the model captures underlying traffic behavior in the transformed feature space. This is noteworthy, given that many confounding variables, such as weather conditions, accidents, and construction, were not included as predictors. Despite these limitations, the model can explain most of the systematic variation in traffic behavior using only time-related patterns and location-specific indicators.

8.5.3 Considerations Regarding MAPE

The model does produce a relatively high Mean Absolute Percentage Error (MAPE), especially on the test set, compared to RMSE and MAE. While this may initially appear concerning, it is largely due to how MAPE behaves when actual values are relatively small.

MAPE expresses error as a percentage of the true observed value. When roads are naturally unoccupied, small absolute errors translate into extremely large percentage errors. For example, an absolute error of 100 vehicles when the true traffic count is 20 vehicles results in a percentage error

exceeding 500% (e.g., | 20 – 120 | = 100/20 = 5 x 100% = 500%). Therefore, because sensor data inherently returns a number of instances when roadway demand is low, such as overnight and early-morning periods, many observations in the data inadvertently raised the aggregated MAPE by a significant amount.

As a result, RMSE and MAE provide a more reliable and meaningful assessment of model performance in this context.

8.6 Interpretation of Coefficients & Features

To interpret the predictive behavior of the Elastic Net model, the estimated coefficients were examined to identify the variables most strongly associated with traffic volume **(Figure 29)**.

```
Intercept: 379.15681468191826
```

	feature	coefficient	abs_coefficient
0	lag_1h	253.134517	253.134517
1	lag_24h	133.039400	133.039400
1878	hour_cos	-92.394115	92.394115
2	lag_168h	88.072122	88.072122
1877	hour_sin	-84.300117	84.300117
1870	direction_SB-MAIN	20.819642	20.819642
1871	direction_SB-SER	-19.535520	19.535520
1866	direction_NB-MAIN	18.178957	18.178957
1867	direction_NB-SER	-15.837402	15.837402
1876	is_weekend	-7.417915	7.417915
1700	segment_id_251949	6.658098	6.658098
1865	direction_NB - SER	-6.262645	6.262645
1879	dow_sin	4.172448	4.172448
1881	month_sin	-4.106623	4.106623
1873	direction_WB-MR	4.001684	4.001684
1880	dow_cos	-3.578379	3.578379
1875	direction_WB-SR	-3.275013	3.275013
1864	direction_NB	3.175163	3.175163
1413	segment_id_154454	2.445095	2.445095

Figure 29 Elastic Net Coefficients

The model intercept was ~379 vehicles, representing the baseline predicted traffic volume when all other predictor variables are centered at zero.

Several features can be deemed as significantly influential in predicting traffic levels across NYC. The most influential predictor was the one-hour-lag variable, with a coefficient of ~253. This implies that, holding all else constant, a one-vehicle increase in traffic volume during the previous hour increases the predicted traffic volume in the current hour by 253 vehicles (and vice versa). This relationship indicates that roadway demand across NYC evolves gradually rather than abruptly from one hour to the next.

Additional lag variables capturing longer-term periodic dependencies also displayed strong predictive influence. The previous-day and previous-week lags had coefficients of ~133 and ~88, respectively. This

demonstrates that traffic behavior tends to repeat across daily and weekly cycles. These results align with expected commuting behavior, in which roadway travel tends to recur at similar times across consecutive days and weeks.

The coefficients for the cyclical periodic features indicate how traffic levels change at different points in the hourly, daily, and monthly cycle relative to the baseline. Specifically, the sine and cosine transformations for say, the hour, are calculated by mapping each hour (0-23) onto a unit circle using sin/cosine(2*pi*hour/24), allowing the model to represent the time of the day as a continuous cyclical pattern rather than on a linear scale. In interpreting the weights, the negative coefficient on hour_cos suggests that at certain times of day (e.g., late night and early morning periods), there is a strong association with lower traffic volumes relative to the average hour. Furthermore, the combination of hourly sine and cosine terms allows the model to identify peak periods, such as morning and evening rush hours, when the predicted traffic increases. Although less interpretable individually, these results prove robust and practical when analyzed in coordination with our EDA.

Finally, directional and certain segment roadway variables also contributed meaningfully to traffic prediction. Positive coefficients for certain travel directions (e.g., northbound or southbound) suggest systematic differences in traffic volume by roadway orientation. Moreover, a handful of coefficients for certain roadway segments suggest that sensors along these roads record higher traffic volumes relative to the baseline.

Overall, our coefficient analysis indicates that traffic volume in New York City is strongly driven by short-term persistence, hourly periodicity, and directional and localized roadway characteristics, all of which are effectively captured by applying our fine-tuned Elastic Net modeling framework (Figure 30).

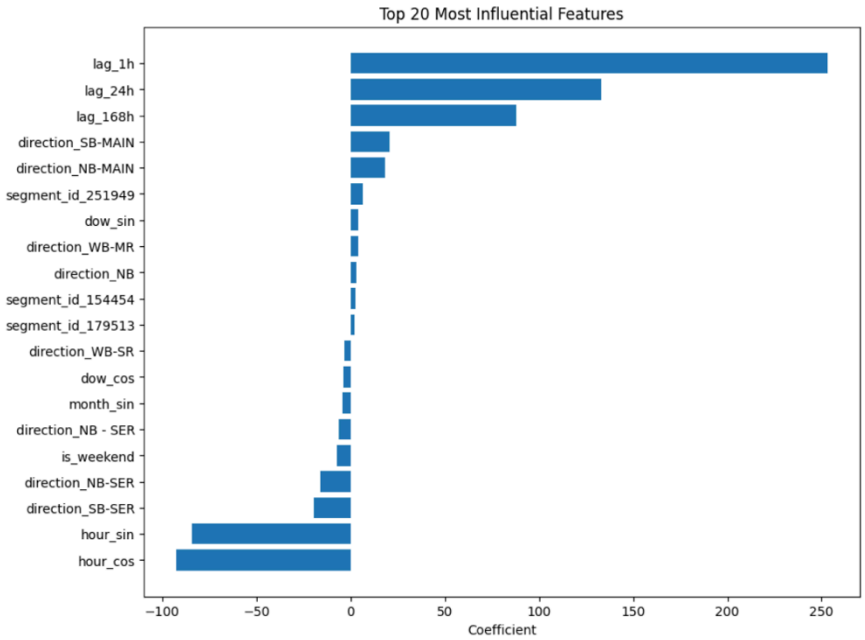

Figure 30 Feature contribution

8.7 Diagnostic Plots

To further evaluate model performance and interpretability, several diagnostic plots were analyzed.

8.7.1 Predicted vs Actual Traffic Volume

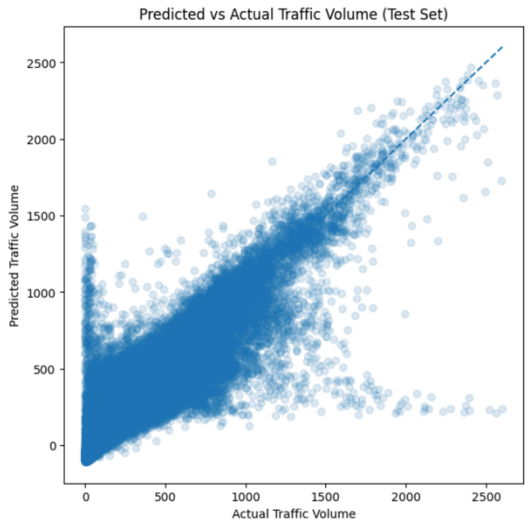

Figure 31 Predicted vs Actual Traffic Volume

This scatter plot compares traffic predictions against actual traffic volume on the test dataset. In it, each data point represents a single hourly observation, while the diagonal line indicates perfectly accurate predictions **(Figure 31)**.

In analyzing the plot, we see a high concentration of points near the diagonal, particularly for low to moderate traffic volumes. Thus, the model performs well when predicting typical NYC traffic scenarios, where most observations occur.

However, the plot also reveals a consistent pattern of underestimation at higher-than-usual traffic volumes, particularly above roughly 1000 vehicles per hour. Quite noticeably, a handful of these points fall below the diagonal line, indicating that the model struggles to fully capture the relationship between the predictors and the response when roadways are extremely congested. These spikes are likely caused by external factors, such as accidents, weather, or other events, that are not specifically captured by our model.

From a practical perspective, this implies that the model is well-suited to identifying general traffic patterns but may underpredict rare and extremely high congestion volumes.

8.7.2 Residual Distribution

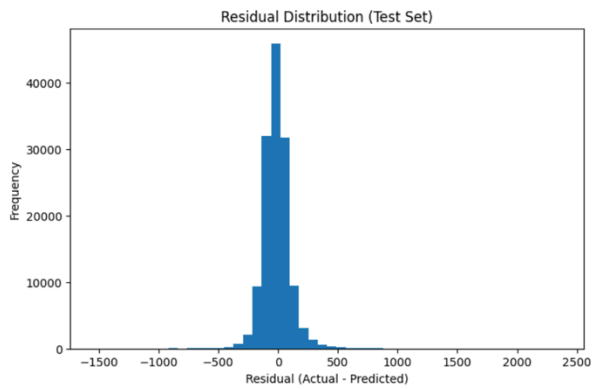

Figure 32 Residual Distribution

The above histogram shows the distribution of prediction errors (actual minus predicted traffic volume) in the test data. Ideally, the residuals should be centered around zero and normally distributed **(Figure 32)**.

The observed distribution appears tightly concentrated around zero, with an ideal symmetric shape, indicating that the model doesn't systematically overestimate or underestimate traffic volumes. On average, the model's predictions are relatively unbiased to either extreme.

However, a small number of observations do appear in the tails of the distribution, corresponding to larger prediction errors. Again, these are likely associated with irregular and unpredictable traffic events that are not well captured using solely the predictor set available on hand.

9. Business Recommendations

This analysis demonstrates IoT traffic data can do more than monitor conditions in NYC. Traffic patterns follow consistent cycles and are influenced by recent levels, allowing proactive use. Agencies could predict traffic increases and take preventive actions, such as adjusting signals or informing drivers.

Not all roads behave the same. Clustering revealed distinct groups ranging from residential streets to high-volume corridors, suggesting that strategies should be customized for each type, with high-traffic areas requiring more monitoring.

The findings inform long-term planning by prioritizing high-traffic areas for improvements such as signal upgrades or capacity boosts, ensuring resources are well allocated. IoT data can also detect unusual conditions, signaling disruptions like accidents or events, enabling faster response.

In practical terms, this project shows that NYC can use IoT traffic data not just to observe congestion, but to anticipate it, prioritize where to intervene, and make more disciplined operational and infrastructure decisions.

10. Conclusion

This project showcases how IoT-generated traffic data can improve the understanding and prediction of traffic patterns in a large city like New York. The analysis reveals that traffic behavior follows distinct temporal and spatial patterns that can be modeled with high accuracy. Using exploratory analysis, clustering, anomaly detection, and predictive modeling, the study goes beyond simply describing traffic to forecasting it, supporting better decision-making.

In practice, its main benefit is enabling more informed and timely operational and business decisions. The findings suggest that traffic conditions can often be forecasted, allowing agencies to respond proactively rather than reactively to congestion. Identifying busy corridors and grouping similar roads offers a structured way to prioritize operational efforts and long-term planning.

Nonetheless, the analysis also has limitations. While the model performs well under normal conditions, it struggles to capture extreme congestion caused by external factors such as weather, accidents, or special events. Future work could incorporate additional data sources to improve the detection of these disruptions.

Overall, this project demonstrates how integrating IoT data with machine learning techniques fosters a more data-driven approach to traffic management. As a result, IoT analytics can greatly enhance urban transportation system management.

11. References

Department of Transportation (DOT). (2026, January 30). *Traffic Volume Counts | NYC Open Data*. Data.cityofnewyork.us. https://data.cityofnewyork.us/Transportation/Traffic-Volume-Counts/btm5-ppia/about_data